Los Sabrossos

Contents

1. Math

1.1. Sieve of Eratosthenes

```
const int N = 1e7;
int sieve[N];
void make(){
    for(int i = 0; i < N; i++) sieve[i] = 1;
    sieve[0] = sieve[1] = 0;
    for (int i = 2; i < N; i++)
    {
        if(sieve[i]){
            for (int j = i*i; j < N; j+=i)
            {
                sieve[j] = 0;
            }
        }
    }
}
```

1.2. Binary Exponentiation

```
long long pote(long long a, long long b, long long mod){
    long long ans = 1ll;
    while(b){
        if(b&1) ans = (ans * a)%mod;
        a = (a * a)%mod;
        b>>=1;
    }
    return ans;
}
// a^-1 = a^mod-2 = pote(a,mod-2);
```

1.3. Fibonacci Log(n)

```
long long int fibo(int n) {
    long long int a, b, x, y, tmp;
    a = x = tmp = 0;
    b = y = 1;
    while (n != 0)
        if (n % 2 == 0) {
            tmp = a * a + b * b;
            b = b * a + a * b + b * b;
            a = tmp;
            n = n / 2;
        } else {
            tmp = a * x + b * y;
            y = b * x + a * y + b * y;
            x = tmp;
            n = n - 1;
        }
    // x = fibo(n-1)
    return y; // y = fibo(n)
}
```

1.4. Modular Inverse

```
const long long N = 1e6 + 10;
const int MOD = 1e9+7;
long long fact[N];
long long invfact[N];
void init(){
    fact[0] = 1;
    for(long long i = 1; i < N; i++)
        fact[i] = fact[i-1] * i % MOD;

    invfact[N-1] = pote(fact[N-1], MOD-2);

        for(long long i = N-2; i >= 0; i--)
        invfact[i] = invfact[i+1] * (i+1) % MOD;
}

long long nCk(long long n, long long k){
    if(k < 0 || k > n) return 0;
    return fact[n] * invfact[k] % MOD * invfact[n-k] % MOD;
}
```

1.5. Pollard Rho Rabin Miller

```
map<long long, long long> fact;

long long mcd(long long a, long long b) {
    a = abs(a);
    b = abs(b);
    while (a != 0) {
        long long r = b % a;
        b = a;
        a = r;
    }
    return b;
}
//Rabin Miller
bool primo(long long n) {
    if (n < 2) return false;
    if (n == 2) return true;
    long long D = n - 1, S = 0;
    while (D % 2 == 0) {
        D /= 2;
        S++;
    }
    static const long long STEPS = 16ll;
    for(long long pasos = 0; pasos < STEPS; pasos++){
        const long long A = 1 + rand() % (n - 1);
        long long M = pote(A, D, n);
        if (M == 1 || M == (n - 1)) goto next;
        for(long long k = 0; k < S-1; k++){
            M = ((M) * M) % n;
            if (M == (n - 1)) goto next;
        }
        return false;
    next:;
    }
    return true;
}

//Rho de pollard
long long factor(long long n) {
    static long long A, B;
    A = 1 + rand() % (n - 1);
    B = 1 + rand() % (n - 1);
    #define fun(x) (((x) * (x + B)) % n + A)

    long long x = 2, y = 2, d = 1;
    while (d == 1 || d == -1) {
        x = fun(x);
        y = fun(fun(y));
        d = mcd(x - y, n);
    }
    return abs(d);
}

void factorize(long long n){
    assert(n > 0);
    while (n > 1 && !primo(n)) {
        long long fx;
        do {
            fx = factor(n);
        } while (fx == n);

        n /= fx;
        factorize(fx);

        for (auto &it : fact) {
            while (n % it.first == 0) {
                n /= it.first;
                it.second++;
            }
        }
    }
    if (n > 1) fact[n]++;
}
```

2. Geometry

2.1. Convex Hull

```
// Devuelve sentido horario
vector<point> hull(vector<point> p)
```

```
{
    int n = p.size();
        vector<point> h;
        sort(p.begin(),p.end());
        for(int i = 0; i < n; i++){
                while(h.size() >= 2 and p[i].left(h[h.size () - 2], h.back()) < 0){
                        h.pop_back();
                }
                h.push_back(p[i]);
        }
        h.pop_back();
        int k = h.size();
        for(int i = n-1; i >= 0; i--)
        {
                while(h.size() >= k + 2 and p[i].left(h[h.size() - 2], h.back()) < 0){
                        h.pop_back();
                }
                h.pb(p[i]);
        }
        h.pop_back();
        return h;
}
```

2.2. Point

```
struct point
{
        int x, y;
        point() {}
        point(int x, int y): x(x), y(y) {}
        point operator -(point p) {return point(x - p.x, y - p.y);}
        point operator +(point p) {return point(x + p.x, y + p.y);}
        int sq() const{return x * x + y * y;}
        double abs() const {return sqrt(sq());}
        int operator ^(point p) {return x * p.y - y * p.x;}
        int operator *(point p) {return x * p.x + y * p.y;}
        point operator *(int a) {return point(x * a, y * a);}
        bool operator <(const point& p) const {return x == p.x ? y < p.y : x < p.x;}
        int left(point a, point b) {
        return ((b - a) ^ (*this - a)); // 0, -, +
    }
};
ostream& operator<<(ostream& os, const point& p) {
    return os << "(" << p.x << "," << p.y << ")\n";
}


void polarSort(vector<point>& v) {
  sort(v.begin(), v.end(), [] (point a, point b) {
    const point origin{0, 0};
    bool ba = a < origin, bb = b < origin;
    if (ba != bb) { return ba < bb; }
    return (a^b) > 0;
  });
}
```

2.3. Closest Pair of Points Lineal

```
struct pt {
    ll x, y;
    bool operator == (const pt & o) const {
        return x == o.x and y == o.y;
    }
};

struct CustomHashPoint {
    size_t operator()(const pt & p) const {
        static
        const uint64_t C =
            chrono::steady_clock::now().time_since_epoch().count();
        uint64_t x = p.x;
        uint64_t y = p.y;
        x ^= x >> 30;
        x *= 0xbf58476d1ce4e5b9;
        x ^= x >> 27;
        x *= 0x94d049bb133111eb;
        x ^= x >> 31;
        y ^= y >> 30;
        y *= 0xbf58476d1ce4e5b9;
        y ^= y >> 27;
        y *= 0x94d049bb133111eb;
```

```
        y ^= y >> 31;
        return C ^ (x + (y << 1));
    }
};

ll dist2(pt a, pt b) {
    ll x = a.x - b.x;
    ll y = a.y - b.y;
    return x * x + y * y;
}

pair < int, int > closest_pair(vector < pt > p) {
    int n = p.size();
    unordered_map < pt, int, CustomHashPoint > mp;
    // puntos repetidos
    for (int i = 0; i < n; i++) {
        if (mp.count(p[i]))
            return {
                mp[p[i]],
                i
            };
        mp[p[i]] = i;
    }
    mt19937 rng(
        chrono::steady_clock::now().time_since_epoch().count()
    );
    auto rnd = [ & ]() {
        return uniform_int_distribution < int > (0, n - 1)(rng);
    };
    ll d2 = dist2(p[0], p[1]);
    pair < int, int > ans = {
        0,
        1
    };
    auto upd = [ & ](int i, int j) {
        ll nd = dist2(p[i], p[j]);
        if (nd < d2) {
            d2 = nd;
            ans = {
                i,
                j
            };
        }
    };
    // conseguir una distancia inicial razonable
    for (int it = 0; it < 20; it++) {
        int i = rnd();
        int j = rnd();
        while (i == j) j = rnd();
        upd(i, j);
    }
    ll d = sqrtl((long double) d2) + 1;
    unordered_map < pt, vector < int > , CustomHashPoint > grid;
    grid.reserve(2 * n);
    for (int i = 0; i < n; i++) {
        grid[{
            p[i].x / d,
            p[i].y / d
        }].push_back(i);
    }
    for (auto & [c, v]: grid) {
        // misma celda
        for (int i = 0; i < (int) v.size(); i++) {
            for (int j = i + 1; j < (int) v.size(); j++) {
                upd(v[i], v[j]);
            }
        }
        // celdas vecinas
        for (int dx = -1; dx <= 1; dx++) {
            for (int dy = -1; dy <= 1; dy++) {
                if (dx == 0 and dy == 0) continue;
                pt to = {
                    c.x + dx,
                    c.y + dy
                };
                auto it = grid.find(to);
                if (it == grid.end()) continue;
                for (int i: v) {
                    for (int j: it -> second) {
                        upd(i, j);
                    }
                }
            }
        }
    }
    return ans;
```

```
}
```

2.4. Closest Pair of Points Nlogn

```
struct pt {
    ll x, y;
    int id;
};

ll dist2(pt a, pt b) {
    ll x = a.x - b.x;
    ll y = a.y - b.y;
    return x * x + y * y;
}

pair < int, int > closest_pair(vector < pt > p) {
    int n = p.size();
    sort(p.begin(), p.end(), [](pt a, pt b) {
        if (a.x != b.x) return a.x < b.x;
        return a.y < b.y;
    });
    set < pair < ll, int >> s; // {y,id}
    ll d2 = dist2(p[0], p[1]);
    pair < int, int > ans = {
        p[0].id,
        p[1].id
    };
    int l = 0;
    for (int i = 0; i < n; i++) {
        ll d = sqrtl((long double) d2) + 1;
        // sacar puntos que ya est n demasiado lejos en x
        while (l < i and p[i].x - p[l].x >= d) {
            s.erase({
                p[l].y,
                p[l].id
            });
            l++;
        }
        // buscar puntos con y cerca
        auto it = s.lower_bound({
            p[i].y - d,
            -1
        });
        while (it != s.end() and it -> first <= p[i].y + d) {
            int j = it -> second;
            ll nd = dist2(p[i], p[j]);
            if (nd < d2) {
                d2 = nd;
                ans = {
                    p[i].id,
                    p[j].id
                };
                d = sqrtl((long double) d2) + 1;
            }
            it++;
        }
        s.insert({
            p[i].y,
            p[i].id
        });
    }
    return ans;
}
```

3. Strings

3.1. KMP

```
vector<int> kmp(string s){
    int n = s.size();
    vector<int> pi(n);
    for(int i = 1; i < n; i++){
        int j = pi[i-1];
        while(j > 0 and s[i] != s[j]){
            j = pi[j-1];
        }
        if(s[i] == s[j]){
           j++;
```

```
        }
        pi[i] = j;
    }
    return pi;
}
//Pattern Matching
vector<int> ocurrencia(string texto, string patron) {
    vector<int> P = kmp(patron);
    int k = 0;
    vector<int> M;
    repl(i,0,texto.size()) {
        while (k > 0 and patron[k] != texto[i]) k = P[k - 1];
        if (patron[k] == texto[i]) k++;
        if (k == patron.size()) {
            M.pb(i - k + 1);
            k = P[k - 1];
        }
    }
    return M;
}

//Automata
void genAut(string &s) {
    int n = s.size();

    vector<vector<int>> aut(n+1, vector<int>(26));
    vector<int> pi = kmp(s);

    for(int i = 0 ; i < n; i++){
        aut[i][s[i]-'a'] = i+1;
    }

    for(int i = 0; i < n+1; i++){
        for(int c = 0; c < 26; c++){
            if (aut[i][c]) continue;
            if (i < n and s[i] == 'a' + c) aut[i][c] = i+1;
            else if (i > 0) aut[i][c] = aut[pi[i-1]][c];
            else aut[i][c] = 0;
        }
    }
}
```

3.2. Trie

```
const int E = 26;

struct AC {
    int nodes;
    vector<array<int, E>>go;
    vector<bool>terminal;

    AC(){
        add_node();
        nodes = 1;
    }

    void add_node(){
        terminal.emplace_back();
        go.emplace_back();
    }

    void insert(string &s){
        int pos =  0;
        for(char ch : s){
            int c = ch - '0';
            if(go[pos][c] == 0){
                go[pos][c] = nodes++;
                add_node();
            }
            pos = go[pos][c];
        }
        terminal[pos] =  true;
    }

};
```

3.3. Suffix Array

```
vector<int> suffix_array(string s) {
    s += char(0);
    const int n = s.size();
```

```
    vector<int>a(n);
    iota(a.begin(), a.end(), 0);
    sort(a.begin(), a.end(), [&] (int i, int j){
        return s[i] < s[j];
    });
    vector<int> mapping(n);
    mapping[a[0]] = 0;
    for(int i = 1; i < n; i++){
        mapping[a[i]] = mapping[a[i-1]]+(s[a[i-1]] != s[a[i]]);
    }
    int len = 1;
    vector<int>sbs(n);
    vector<int>head(n);
    vector<int>new_mapping(n);
    while(len < n){
        for(int i = 0; i<n; i++) sbs[i] = (a[i] + n - len) % n;
        for(int i = n-1; i>=0; i--) head[mapping[a[i]]] = i;
        for(int i = 0; i < n; i++){
            int x = sbs[i];
            a[head[mapping[x]]++] = x;
        }
        new_mapping[a[0]] = 0;
        for(int i = 1; i < n; i++){
            if(mapping[a[i-1]] != mapping[a[i]]){
                new_mapping[a[i]] = new_mapping[a[i-1]]+1;
            }
            else{
                int pre = mapping[(a[i-1] + len)%n];
                int cur = mapping[(a[i]+len) % n];
                new_mapping[a[i]] = new_mapping[a[i-1]] + (pre!=cur);
            }
        }
        len <<= 1;
        swap(mapping, new_mapping);
    }
    return vector<int>(a.begin()+1, a.end());//ignorar a[0] que es el centinela
}

vector<int> lcp_array(vector<int>&a, string s){
    const int n = s.size();
    vector<int> rank(n);
    for(int i = 0; i < n; i++) rank[a[i]] = i;
    int k = 0;
    vector<int>lcp(n);
    for(int i = 0; i < n; i++){
        if(rank[i] == n-1){
            k = 0;
            continue;
        }
        int j = a[rank[i]+1];
        while(i+k < n and j+k < n and s[i+k] == s[j+k]) k++;
        lcp[rank[i]] = k;
        if(k) k--;
    }
    lcp.pop_back();
    return lcp;
}

void solve(){
    //Numero de distintos substr en un string n*(n+1)/2 - sumatoria del LCP
    //verificar si un string es substring de otro con BS en SA
    //encontrar el longest common substring en 2 strings(ESTA IMPLEMENTACION)
    string s, t;
    cin >> s >> t;
    if(s.size() > t.size()){
        swap(s, t);
    }
    int n = s.size();
    int m = t.size();
    string st = s+'#'+t;
    vector<int>sa = suffix_array(st);
    vector<int>lcp =lcp_array(sa, st);
    // print(sa);
    // print(lcp);
    vector<int>tr(n+m);
    repl(i,0,n+m){
        if(sa[i] >=n and sa[i+1]>=n){
            tr[i] = 0;
            continue;
        }
        if(sa[i] < n and sa[i+1] < n){
            tr[i] = 0;
            continue;
        }
        int mn = min(sa[i], sa[i+1]);
        int tam1 = mn + lcp[i];
```

```
        // dbg(tam1);
        tr[i] = lcp[i];
        if(tam1 > n){
            dbg(tam1);
            tr[i] = lcp[i] - (tam1-n);
        }
    }
    // print(tr);
    int mx = *max_element(tr.begin(), tr.end());
    repl(i,0,n+m){
        // dbg(tr[i]);
        if(tr[i] == mx){
            string ans = st.substr(sa[i], mx);
            cout << ans << '\n';
            return;
        }
    }

}
//Sabrossus
```

3.4. Aho Corasick

```
//AhoCorasick para saber si un string s existe en un texto T
const int E = 26;
const int N = 1e6+10;
int cur;
int nodoTerminal[N];//se guarda el nodo terminal del string i
set<int>st;//se guarda que nodos son terminales que aparecen en el texto T

struct AC {
    int nodes;
    vector<int>suf, super;
    vector<array<int, E>>go;
    vector<bool>terminal;
    AC(){
        add_node();
        nodes = 1;
    }
    void add_node(){
        terminal.emplace_back();
        go.emplace_back();
        super.emplace_back();
        suf.emplace_back();
    }

    void insert(string &s){
        int pos =  0;
        for(char ch : s){
            int c = ch - 'a';
            if(go[pos][c] == 0){
                go[pos][c] = nodes++;
                add_node();
            }
            pos = go[pos][c];
        }
        terminal[pos] =  true;
        nodoTerminal[cur] = pos;
    }

    void build(){
        queue<int>Q;
        Q.emplace(0);
        while(!Q.empty()){
            int u = Q.front();
            Q.pop();
            super[u] = (suf[u] == 0 or terminal[suf[u]] ? suf[u] : super[suf[u]]);
            for(int c = 0; c < E; c++){
                if(go[u][c]){
                    int v = go[u][c];
                    Q.emplace(v);
                    suf[v] = u == 0 ? 0 : go[suf[u]][c];
                }
                else{
                    go[u][c] = u == 0 ? 0 :go[suf[u]][c];
                }
            }
        }
    }
};

void mark(int u, AC &ac){
    if(u == 0) return;
    mark(ac.super[u], ac);
```

```
    if(ac.terminal[u]){
        st.insert(u);
        ac.terminal[u] = false;
    }
    ac.super[u] = 0;
}

void solve(){
    int n;
    cin >> n;
    AC ac;
    repl(i,0,n){
        string s;
        cin >> s;
        cur = i;
        ac.insert(s);
    }
    string t; //texto grande
    cin >> t;
    ac.build();
    int p = 0;
    for(auto x : t){
        int c = x - 'a';
        p = ac.go[p][c];
        mark(p, ac);
    }
    repl(i,0,n){
        if(st.count(nodoTerminal[i])){
            cout << "YES\n";
        }
        else{
            cout << "NO\n";
        }
    }
}

//-------------------------------------------------------------------------

//AhoCorasick para Frecuencias
const int E = 26;
const int N = 1e6+10;
int nodoTerminal[N];
int cur;
struct AC {
    int nodes;
    vector<int>suf, freq, by_level;
    vector<bool>terminal;
    vector<array<int,E>>go;

    AC(){
        add_node();
        nodes = 1;
    }
    void add_node(){
        terminal.emplace_back();
        suf.emplace_back();
        go.emplace_back();
        freq.emplace_back();
    }

    void insert(string &s){
        int pos = 0;
        for(char ch:s){
            int c = ch - 'a';
            if(go[pos][c] == 0){
                go[pos][c] = nodes++;
                add_node();
            }
            pos = go[pos][c];
        }
        terminal[pos] = true;
        nodoTerminal[cur] = pos;
    }

    void build(){
        queue<int>Q;
        Q.emplace(0);
        while(!Q.empty()){
            int u = Q.front();
            Q.pop();
            by_level.emplace_back(u);
            for(int c = 0; c < E; c++){
                if(go[u][c]){
                    int v = go[u][c];
                    Q.emplace(v);
                    suf[v] = u == 0 ? 0 : go[suf[u]][c];
```

```
                    }
                    else{
                        go[u][c] = u == 0 ? 0 : go[suf[u]][c];
                    }
                }
            }
        }
};

void solve(){
    int n;
    cin >> n;
    AC ac;
    repl(i,0,n){
        string s;
        cin >> s;
        cur = i;
        ac.insert(s);
    }
    string t;
    cin >> t;
    ac.build();
    int p = 0;
    for(char ch:t){
        int c = ch-'a';
        p = ac.go[p][c];
        ac.freq[p]++;
    }
    for(int i = (int)ac.by_level.size()-1; i > 0; i--){
        int x = ac.by_level[i];
        ac.freq[ac.suf[x]] += ac.freq[x];
    }
    repl(i,0,n){
        int termi = nodoTerminal[i];
        cout << ac.freq[termi] << '\n';
    }
}
```

3.5. Hashing

```
struct Hash {
        long long P=1777771,MOD[2],PI[2];
        vector<long long> h[2],pi[2];
        Hash(string& s){
                MOD[0]=999727999;
                MOD[1]=1070777777;
                PI[0]=325255434;
                PI[1]=10018302;
                for(int k = 0; k < 2; k++) {
                        h[k].resize(s.size()+1);
                        pi[k].resize(s.size()+1);
                }
                for(int k = 0; k < 2; k++){
                        h[k][0] = 0;
                        pi[k][0] = 1;
                        long long p = 1;
            for(int i = 1; i < s.size()+1; i++){
                                h[k][i] = (h[k][i-1]+p*s[i-1])%MOD[k];
                                pi[k][i] = (1LL*pi[k][i-1]*PI[k])%MOD[k];
                                p = (p*P)%MOD[k];
                        }
                }
        }
        long long get(int s, int e){
                long long h0 = (h[0][e]-h[0][s]+MOD[0])%MOD[0];
                h0 = (1LL*h0*pi[0][s])%MOD[0];
                long long h1 = (h[1][e]-h[1][s]+MOD[1])%MOD[1];
                h1 = (1LL*h1*pi[1][s])%MOD[1];
                return (h0<<32)|h1;
        }
};
```

3.6. Manacher

```
const int N = 1e5 + 10;
int odd[N]; //odd[i] = max odd palindrome centered on i
int even[N]; //even[i] = max even palindrome centered on i
void manacher(string & s) {
    int l = 0, r = -1, n = s.size();
    for (int i = 0; i < n; i++) {
        int k = i > r ? 1 : min(odd[l + r - i], r - i);
```

```
        while (i + k < n and i - k >= 0 and s[i + k] == s[i - k]) k++;
        odd[i] = k--;
        if (i + k > r) l = i - k, r = i + k;
    }
    l = 0;
    r = -1;
    for (int i = 0; i < n; i++) {
        int k = i > r ? 0 : min(even[l + r - i + 1], r - i + 1);
        k++;
        while (i + k <= n and i - k >= 0 and s[i + k - 1] == s[i - k]) k++;
        even[i] = --k;
        if (i + k - 1 > r) l = i - k, r = i + k - 1;
    }
}
```

4. Bit Manipulation

4.1. Binary Representation

```
string tobit(long long x) {
    string a(64, '0');
    for (int i = 63; i >= 0; --i) {
        if (x & 1) a[i] = '1';
        x >>= 1;
    }
    return a;
}
```

4.2. Bit Mask

```
for(int mask = 0; mask < (1 << n); mask++) { //(1 << n) = O(2^n)
    for(int i = 0; i < n; i++) { // O(n)
    //verificamos si el i-esimo bit esta encendido
        if(mask & (1 << i)) {
            cout << v[i] << " ";
        }
    }
}
```

5. Data Structures

5.1. Segment Tree

```
const int N = 1e5;

struct info {
    int num;
};

info ST[4 * N];

info ope(info u, info v) {
    return {u.num + v.num};
}

void build(const vector<int> &v, int in = 1, int L = 0, int R = n - 1) {
    if (L == R) {
        ST[in] = {v[L]}; // Asignamos el valor desde el vector
    } else {
        int mid = (L + R) >> 1;
        build(v, 2 * in, L, mid);
        build(v, (2 * in) + 1, mid + 1, R);
        ST[in] = ope(ST[2 * in], ST[(2 * in) + 1]);
    }
}

void update(int pos, int val, int in = 1, int L = 0, int R = n - 1) {
    if (L == R) {
        ST[in].num = val;
    } else {
        int mid = (L + R) >> 1;
        if (pos <= mid) {
            update(pos, val, 2 * in, L, mid);
```

```
        } else {
            update(pos, val, (2 * in) + 1, mid + 1, R);
        }
        ST[in] = ope(ST[2 * in], ST[(2 * in) + 1]);
    }
}

info get(int l, int r, int in = 1, int L = 0, int R = n - 1) {
    if (r < L or R < l) return {0}; // Elemento neutro
    if (l <= L and R <= r) return ST[in];

    int mid = (L + R) >> 1;
    info left = get(l, r, 2 * in, L, mid);
    info right = get(l, r, (2 * in) + 1, mid + 1, R);
    return ope(left, right);
}


// Llamadas simples y limpias:
// build(v);              // Construye con el vector
// update(2, 10);         // Cambia el valor en pos 2 a 10
// info ans = get(1, 3); // Obtiene la respuesta en el rango [1, 3]
// v = {1 2 3 4 5}
// get(1, 0, n-1, 1, 3) = 2 + 3 + 4 = 9
```

5.2. BIT Fenwick Tree

```
struct BIT{
    int n;
    vector<int> bit;

    BIT(int _n){
        n =_n;
        bit.assign(n+1,0);
    }

    void add(int i, int val){
        while(i<=n){
            bit[i] += val;
            i += (i&(-i));
        }
    }

    int sum(int i){
        int ans = 0;
        while(i > 0){
            ans += bit[i];
            i -= (i&(-i));
        }
        return ans;
    }

    int query(int l, int r){
        return sum(r) - sum(l-1);
    }
};
//se debe utilizar con index 1
//Sabrossus
```

5.3. Bit 2D

```
struct BIT_2D{
    vector<vector<int>> bit;
    int n,m;

    BIT_2D(int x, int y){
        bit.assign(x+1,vector<int> (y+1,0));
        n = x;
        m = y;
    }

    void update(int x, int y, int val){
        for(int i = x ; i <= n ; i+=i&(-i)){
            for(int j= y; j <=m ; j+=j&(-j)){
                bit[i][j] += val;
            }
        }
    }

    int sum(int x, int y){
        int ans = 0;
```

```
        for(int i = x ; i >0 ; i-=i&-i){
            for(int j= y; j >0 ; j-=j&-j){
                ans += bit[i][j];
            }
        }
        return ans;
    }

    int query(int x1, int y1, int x2, int y2){
        return sum(x2,y2) + sum(x1-1,y1-1) - sum(x1-1,y2) - sum(x2,y1-1);
    }
};

void solve(){
    int n,q;
    cin >> n >> q;
    BIT_2D bit(n,n);
    vector<vector<int>> mt(n+1,vector<int>(n+1,0));
    for(int i = 1 ;i  <= n; i ++){
        for(int j = 1 ; j <= n ;j++){
            char uwu;
            cin >> uwu;
            if(uwu=='.') continue;
            mt[i][j] = 1;
            bit.update(i,j,1);
        }
    }
    while(q--){
        int tipo;
        cin >> tipo;
        if(tipo == 1){
            int x,y;
            cin >> x >> y;
            if(mt[x][y]){
                bit.update(x,y,-1);
            }
            else{
                bit.update(x,y,1);
            }
            mt[x][y] ^= 1;
        }
        else{
            int x1,y1,x2,y2;
            cin >> x1 >> y1 >> x2 >>y2;
            cout << bit.query(x1,y1,x2,y2) << '\n';
        }
    }
}
//No entiendo w, el dildan lo hizo
```

5.4. Disjoint Set Union

```
struct DSU{
    int num;
    vector<int> parent;
    vector<int> sizes;
    DSU(){};
    DSU(int n){
        init(n);
    }
    void init(int n){
        num = n;
        parent.resize(n+1);
        sizes.resize(n+1);
        //si tienes index 0 inicializar desde 0
        for(int i = 1; i <= n ; i++){
            parent[i] = i;
            sizes[i] = 1;
        }
    }
    int find(int a){
        if(a == parent[a]) return a;
        return parent[a] = find(parent[a]);
    }
    void join(int a, int b){
        int x = find(a);
        int y = find(b);
        if(x == y) return;
        num--;
        if(sizes[x] < sizes[y]){
            parent[x] = y;
            sizes[y] += sizes[x];
        }
        else{
```

```cpp
            parent[y] = x;
            sizes[x] += sizes[y];
        }

    }
};
//Sabrossus
```

5.5. Implicit Treap

```cpp
// Mantiene las posiciones
struct Node {
    int val , pri , sz ;
    Node *l , *r;
    Node ( int v) {
        val = v;
        pri = rand () ;
        sz = 1;
        l = r = nullptr ;
    }
};
using pnode = Node *;
int sz ( pnode t) {
    return t ? t -> sz : 0;
}
void upd (pnode t) {
    if (t) t -> sz = 1 + sz (t -> l) + sz (t -> r);
}
void split ( pnode t , pnode &l , pnode &r , int k) {
    if (! t) {
        l = r = nullptr ;
        return ;
    }
    int leftSize = sz (t -> l);
    if ( leftSize >= k) {
        split (t -> l , l , t ->l , k) ;
        r = t;
    }
    else {
        split (t -> r , t ->r , r , k - leftSize - 1) ;
        l = t;
    }
    upd (t);
}
pnode merge ( pnode l , pnode r) {
    if (! l || !r ) return l ? l : r;

    if (l -> pri > r -> pri ) {
        l -> r = merge (l -> r , r);
        upd (l );
        return l;
    }
    else {
        r -> l = merge (l , r ->l );
        upd (r );
        return r;
    }
}
void insert ( pnode & root , int pos , int val ) {
    pnode a , b;
    split ( root , a , b , pos );
    root = merge ( merge (a , new Node ( val )) , b) ;
}
void erase ( pnode & root , int pos ) {
    pnode a , b , c;
    split ( root , a , b , pos );
    split (b , b , c , 1) ;
    root = merge (a , c) ;
}
void print(pnode t) {
    if (!t) return;

    print(t->l);
    cout << t->val << " ";
    print(t->r);
}
int get(pnode t, int k) {
    if (!t) return -1;

    int leftSize = sz(t->l);

    if (k < leftSize)
        return get(t->l, k);

```

```
    if (k == leftSize)
        return t->val;

    return get(t->r, k - leftSize - 1);
}
```

5.6. Ordered Set

```
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

using namespace std;
using namespace __gnu_pbds;

typedef tree<int, null_type, less<int>, rb_tree_tag, tree_order_statistics_node_update> ordered_set;

void solve() {
    ordered_set st;

    st.insert(10);
    st.insert(30);
    st.insert(20);

    // 1. Elemento en la posicion K (0-indexed, de menor a mayor)
    // find_by_order devuelve un iterador (puntero). Usamos '*' para el valor.
    auto it = st.find_by_order(1);
    if (it != st.end()) {
        int elemento = *it; // Devuelve 20 de forma segura
    }

    // 2. Cuantos elementos son estrictamente menores que X
    int menores_que_25 = st.order_of_key(25); // Devuelve 2 (el 10 y el 20)
}
// Sabrossus
```

5.7. Sparse Table

```
const int N = 1e5;
const int LOG = 20;
int v[N]; //values
int st[N][LOG]; //Sparse Table
int lg[N]; // log values
int n;
void build(){
    for(int i = 0; i < n; i++) st[i][0] = v[i];

    for(int i = 1; i < LOG; i++){
        for(int j = 0; j < n - (1<<i) + 1; j++){
            st[j][i] = min(st[j][i-1],st[j+(1<<(i-1))][i-1]);
        }
    }
    lg[1] = 0;
    for(int i = 2; i <= n;i++) lg[i] = lg[i/2] + 1;
}
int query(int L, int R){
    int k = lg[R-L+1];
    return min(st[L][k],st[R-(1<<k)+1][k]);
}
// v = [5, 2, 4, 7, 1, 3]
// query(1, 4)      [2, 4, 7, 1]
```

5.8. Treap

```
// Ordena los elementos por valor
struct Node {
    int key, pri, sz;
    Node *l, *r;
    Node(int k) {
        key = k;
        pri = rand();
        sz = 1;
        l = r = nullptr;
    }
};

```

```cpp
using pnode = Node*;

int sz(pnode t) {
    return t ? t->sz : 0;
}

void upd(pnode t) {
    if (t) t->sz = 1 + sz(t->l) + sz(t->r);
}

void split(pnode t, pnode &l, pnode &r, int k) {
    if (!t) {
        l = r = nullptr;
        return;
    }

    int leftSize = sz(t->l);

    if (leftSize >= k) {
        split(t->l, l, t->l, k);
        r = t;
    } else {
        split(t->r, t->r, r, k - leftSize - 1);
        l = t;
    }

    upd(t);
}

pnode merge(pnode l, pnode r) {
    if (!l || !r) return l ? l : r;

    if (l->pri > r->pri) {
        l->r = merge(l->r, r);
        upd(l);
        return l;
    } else {
        r->l = merge(l, r->l);
        upd(r);
        return r;
    }
}

void insert(pnode &root, int pos, int val) {
    pnode a, b;
    split(root, a, b, pos);
    root = merge(merge(a, new Node(val)), b);
}

void erase(pnode &root, int pos) {
    pnode a, b, c;
    split(root, a, b, pos);
    split(b, b, c, 1);
    root = merge(a, c);
}

int get(pnode t, int k) {
    if (!t) return -1;

    int leftSize = sz(t->l);

    if (k < leftSize)
        return get(t->l, k);

    if (k == leftSize)
        return t->key;

    return get(t->r, k - leftSize - 1);
}

void print(pnode t) {
    if (!t) return;

    print(t->l);
    cout << t->key << " ";
    print(t->r);
}
```

6. Dynamic Programing

6.1. Fastfibo

```cpp
#include <bits/stdc++.h>
#define ll long long

using namespace std;

const ll MOD = 1e9 + 7;

struct Matrix {
    ll a[2][2];
    Matrix() {
        memset(a, 0, sizeof(a));
    }
};

Matrix multiply(Matrix A, Matrix B) {
    Matrix C;
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            for (int k = 0; k < 2; k++) {
                C.a[i][j] = (C.a[i][j] + A.a[i][k] * B.a[k][j]) % MOD;
            }
        }
    }
    return C;
}

Matrix power(Matrix A, ll n) {
    Matrix R;
    R.a[0][0] = 1;
    R.a[1][1] = 1;
    while (n) {
        if (n & 1)
            R = multiply(R, A);
        A = multiply(A, A);
        n >>= 1;
    }
    return R;
}

ll fibonacci(ll n) {
    if (n == 0) return 0;
    Matrix A;
    A.a[0][0] = 1;
    A.a[0][1] = 1;
    A.a[1][0] = 1;
    A.a[1][1] = 0;
    Matrix R = power(A, n - 1);
    return R.a[0][0];
}

int main() {
    ll n;
    cin >> n;
    cout << fibonacci(n) << '\n';
}
```

7. Flow

7.1. Dinic

```cpp
struct Dinic
{
    const int INF = 1e9;

    struct flowEdge
    {
        int to, rev, f, cap;
    };

    vector<vector<flowEdge>> G;
    vector<int> work, lvl, Q;

    int S, T, nodes;

    Dinic(){}
    Dinic(int n, int s, int t){
        S = s;
        T = t;
        nodes = n;
        G.clear();
        G.resize(n);
```

```cpp
        Q.resize(n);
    }
    void addEdge(int st, int en , int cap){
        flowEdge A = {en, (int)G[en].size(), 0, cap};
        flowEdge B = {st, (int)G[st].size(), 0, 0};
        G[st].pb(A);
        G[en].pb(B);
    }
    int dfs(int v, int f){
        if(v == T or f == 0) return f;
        for(int &i = work[v]; i < G[v].size(); i++){
            flowEdge &e = G[v][i];
            int u = e.to;
            if(e.cap <= e.f or lvl[u] != lvl[v] + 1) continue;
            int df = dfs(u, min(f, e.cap - e.f));
            if(df){
                e.f += df;
                G[u][e.rev].f -= df;
                return df;
            }
        }
        return 0;
    }
    bool bfs(){
        int qt = 0;
        Q[qt++] = S;
        lvl.assign(nodes,-1);
        lvl[S] = 0;
        for(int i = 0; i < qt; i++){
            int u = Q[i];
            for(flowEdge &e : G[u]){
                int v = e.to;
                if(e.cap <= e.f or lvl[v] != -1) continue;
                lvl[v] = lvl[u] + 1;
                Q[qt++] = v;
            }
        }
        return lvl[T] != -1;
    }
    int maxFlow(){
        int flow = 0;
        while(bfs()){
            work.assign(nodes,0);
            while(true){
                int df = dfs(S,INF);
                if(df == 0) break;
                flow += df;
            }
        }
        return flow;
    }

};
```

8. Graph Algorithms

8.1. Dijkstra

```cpp
const int N = 1e5;

int vis[N];
vector<pair<int,int>> G[N];

void dijkstra(int x){
    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> pq;
    pq.push({0,x});
    vis[x] = 0;
    while(!pq.empty()){
        auto y = pq.top();
        pq.pop();
        int u = y.second;
        int cost = y.first;
        for(auto v : G[u]){
            int cur = cost + v.second;
            if(cur < vis[v.first]){
                vis[v.first] = cur;
                pq.push({cur,v.first});
            }
        }
    }
}
```

8.2. Kruskal MST

```
const int N = 1e5;
struct DSU{
    int num;
    vector<int> parent;
    vector<int> sizes;
    DSU(){};
    DSU(int n){
        init(n);
    }
    void init(int n){
        num = n;
        parent.resize(n+1);
        sizes.resize(n+1);
        for(int i = 1; i <= n ; i++){
            parent[i] = i;
            sizes[i] = 1;
        }
    }
    int find(int a){
        if(a == parent[a]) return a;
        return parent[a] = find(parent[a]);
    }
    void join(int a, int b){
        int x = find(a);
        int y = find(b);
        if(x == y) return;
        num--;
        if(sizes[x] < sizes[y]){
            parent[x] = y;
            sizes[y]+=sizes[x];
        }
        else{
            parent[y] = x;
            sizes[x]+=sizes[y];
        }
    }
};
int n;
vector<pair<int,pair<int,int>>> List;
int Kruskal_MST(){
    sort(List.begin(),List.end());
    DSU dsu(n);
    int mst = 0, t = 1;
    for(int i = 0; i < List.size(); i++){
        int w = List[i].first; //weigth
        int a = List[i].second.first; //at
        int b = List[i].second.second; //to
        if(dsu.find(a) != dsu.find(b)){
            dsu.join(a,b);
            mst+=w;
            t++;
        }
        if(t == n) break;
    }
    return mst;
}
```

8.3. Topological Sort Kahn Algorithm

```
const int N = 1e5;

int indegree[N];
vector<int> G[N];
int n;
vector<int> Kahn(){
    queue<int> q;
    vector<int> ans;
    for(int i = 0; i < n; i++){
        if(indegree[i] == 0) q.push(i);
    }
    while (!q.empty()){
        int v = q.front();
        q.pop();
        ans.push_back(v);
        for(int u : G[v]){
            indegree[u]--;
            if(indegree[u] == 0){
                q.push(u);
            }
        }
    }
```

```
    //if(ans.size() != n) grafo no DAG
    return ans;
}
```

9. Tree Algorithms

9.1. Euler Tour

```
const int N = 1e5 + 10;
vector<int> path;
vector<int> G[N];
void EulerTour(int node, int parent){
    path.pb(node);
    for(int u : G[node]){
        if(u != parent){
            EulerTour(u,node);
        }
    }
    path.pb(node);
}

const int N = 1e5 + 10;
int tin[N];
int tout[N];
int flat[N];
int timer = 1;
void EulerTour(int node, int parent){
    tin[node] = timer;
    flat[timer] = node;
    timer++;
    for(int u : G[node]){
        if(u != parent){
            EulerTour(u, node);
        }
    }
    tout[node] = timer;
}
```

9.2. Binary Lifting

```
const int N = 1e5;
const int LOG = 20;
int up[N][LOG];
vector<int> G[N];
//Find parents
void dfs(int u){
    for(int v : G[u]){
        up[v][0] = u;
        for(int j = 1; j < LOG; j++){
            up[v][j] = up[up[v][j-1]][j-1];
        }
        dfs(v);
    }
}
int get_K_parent(int a, int k){
    for(int j = LOG-1; j >= 0; j++){
        if(k&(1<<j)){
            a = up[a][j];
        }
    }
    return a;
}
```

9.3. Centroid Decomposition

```
//EJERCICIO IMPLEMENTACION XENIA AND TREE
//Nodo rojo mas cercano a un nodo v

const int N = 2e5+10;
vector<int>G[N];
int tag[N]; // time of discovery of centrodi
int fat[N]; // father in centroid decomposition
int szt[N]; // size of subtree

int best[N];
```

```
int niv[N];
int dist[30][N]; //dist[nivel][nodo];

void dfs_level(int node, int p, int niv, int prof){
    dist[niv][node] = prof;
    for(auto y : G[node]){
        if(y != p and tag[y] < 0){
            dfs_level(y, node, niv, prof+1);
        }
    }
}
// -----------------CENTROID-------------------------
int calcsz(int node, int p){
    szt[node] = 1;
    for(auto y : G[node]){
        if(y != p and tag[y] < 0){
            szt[node] += calcsz(y, node);
        }
    }
    return szt[node];
}
int ccnt = 0;
void cdfs(int node, int p, int nivel = 0, int sz = -1){
    if(sz < 0) sz = calcsz(node, -1);
    for(auto y : G[node]){
        if(tag[y] < 0 and szt[y] * 2 >= sz){
            szt[node] = 0;
            cdfs(y, p, nivel, sz);
            return;
        }
    }
    tag[node] = ccnt++;
    fat[node] = p;
    niv[node] = nivel;
    dfs_level(node, -1, nivel, 0);
    for(auto y : G[node]){
        if(tag[y] < 0){
            cdfs(y, node, nivel+1);
        }
    }
}

int n;
void centroid(){
    ccnt = 0;
    repl(i,1,n+1){
        tag[i] = -1;
    }
    cdfs(1, -1);
}

//---------------------------------------------

void update(int node, int rojo){
    int nivel = niv[node];
    int cost = dist[nivel][rojo];
    best[node] = min(best[node], cost);
    int p = fat[node];
    if(p > 0){
        update(p, rojo);
    }
}

int improve(int node, int cur){
    if(node <= 0){
        return 1e9;
    }
    int nivel = niv[node];
    int cost = best[node] + dist[nivel][cur];
    int p = fat[node];
    return min(cost, improve(p, cur));
}

void solve(){
    int qq;
    cin >> n >> qq;
    repl(i,0,30){
        repl(j,1,n+1){
            dist[i][j] = -1;
            best[j] = 1e9;
        }
    }
    repl(i,0,n-1){
        int a, b;
        cin >> a >> b;
        G[a].push_back(b);
```

```cpp
            G[b].push_back(a);
        }
        centroid();
        update(1, 1);
        while(qq--){
            int q, node;
            cin >> q >> node;
            if(q == 1){
                update(node, node);
            }
            else{
                int res = best[node];
                int res2 = improve(node, node);
                cout << min(res, res2) << '\n';
            }
        }
}

int main(){
    velocito;
    int t = 1;
    // cin >> t;
    while(t--){
        solve();
    }
    return 0;
}
//Sabrossus
//TC Argentina
```

9.4. Lowest Common Ancestor

```cpp
const int N = 1e5;
const int LOG = 20;
vector<int> G[N];
int depth[N];
int up[N][LOG];
// Assign levels and get parents
void dfs(int u){
    for(int v : G[u]){
        depth[v] = depth[u] + 1;
        up[v][0] = u;
        for(int j = 1; j < LOG; j++){
            up[v][j] = up[up[v][j-1]][j-1];
        }
        dfs(v);
    }
}
int get_lca(int a, int b){
    if(depth[a] < depth[b]) swap(a,b);
    int k = depth[a] - depth[b];
    for(int j = LOG-1; j >= 0; j--){
        if(k & (1<<j)){
            a = up[a][j];
        }
    }
    if(a == b) return a;
    for(int j = LOG-1; j >= 0; j--){
        if(up[a][j] != up[b][j]){
            a = up[a][j];
            b = up[b][j];
        }
    }
    return up[a][0];
}
```

10. Utilities

10.1. Int128

```cpp
// --- LECTURA B SICA ---
__int128 read128() {
    string s;
    cin >> s;
    __int128 n = 0;
    int sign = 1;
    size_t start = 0;

```

```cpp
    if (!s.empty() && s[0] == '-') {
        sign = -1;
        start = 1;
    }

    for (size_t i = start; i < s.length(); ++i) {
        n = n * 10 + (s[i] - '0');
    }

    return n * sign;
}

// --- IMPRESI N B SICA ---
void print128(__int128 n) {
    if (n == 0) {
        cout << 0;
        return;
    }
    if (n < 0) {
        cout << '-';
        n = -n;
    }
    string s;
    while (n > 0) {
        s += (char)('0' + (n % 10));
        n /= 10;
    }
    reverse(s.begin(), s.end());
    cout << s;
}
```

10.2. Merge Sort

```cpp
// Rango de uso: 0-indexed.
// Llamar en el solve como: mergeSort(v, 0, n - 1);
ll ans = 0;

void mergeSort(vector<ll> &v, int low, int high) {
    if (low == high) return;

    int mid = (low + high) >> 1;
    mergeSort(v, low, mid);
    mergeSort(v, mid + 1, high);

    queue<ll> L, H;
    for (int i = low; i < mid + 1; i++) {
        L.push(v[i]);
    }
    for (int i = mid + 1; i < high + 1; i++) {
        H.push(v[i]);
    }

    for (int i = low; i < high + 1; i++) {
        if (L.size() == 0) {
            v[i] = H.front();
            H.pop();
        }
        else if (H.size() == 0) {
            v[i] = L.front();
            L.pop();
        }
        else {
            if (L.front() <= H.front()) {
                v[i] = L.front();
                L.pop();
            }
            else {
                v[i] = H.front();
                H.pop();
                ans += L.size(); // Inversion detectada
            }
        }
    }
}
```

10.3. Preffix Sum 2D

```cpp
//1 indexed
const int N = 1005;
int v[N][N];
int acu[N][N];
```

```cpp
int n, m;

void solve() {
    //limpiar la acu para cada TC

    // Construccion: acumula 1 si el elemento es exactamente 1
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            int val = (v[i][j] == 1 ? 1 : 0);
            acu[i][j] = val + acu[i - 1][j] + acu[i][j - 1] - acu[i - 1][j - 1];
        }
    }

    // Esquinas de consulta:
    // (a, b) -> Superior Izquierda
    // (c, d) -> Inferior Derecha
    int a = 2, b = 2, c = 4, d = 4; // Valores de ejemplo

    int ans = acu[c][d] - acu[a - 1][d] - acu[c][b - 1] + acu[a - 1][b - 1];
    cout << ans << '\n';
}
```

10.4. Random

```cpp
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
// mt19937_64 para 64 bits usar uint64_t o unsigned long long
cout << rng() << endl;
shuffle(v.begin(),v.end(),rng);
```

10.5. Ordered Set

```cpp
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

using namespace __gnu_pbds;

template<class T>
using ordered_set = tree<T, null_type, less<T>,
rb_tree_tag, tree_order_statistics_node_update>;

// find_by_order(k)    elemento en posici n k (0-indexed)
// order_of_key(x)     cantidad de elementos estrictamente
//                     menores que x
```

10.6. Pragma